

Лабораторная работа №1

Линейная регрессия и предобработка данных

Расширенные ответы на все вопросы с формулами и примерами. Подробный разбор каждой строки кода

ЧАСТЬ I: ТЕОРИЯ ЛИНЕЙНОЙ РЕГРЕССИИ

Вопрос 1. Что такое регрессия? Отличие от классификации

Простое объяснение

Представь, что ты риелтор и хочешь предсказать цену квартиры. У тебя есть таблица из сотен квартир: площадь, этаж, район, год постройки — и известная цена каждой. Ты ищешь закономерность: «чем больше площадь — тем выше цена». Это и есть **регрессия** — найти правило, которое по числовым характеристикам объекта предсказывает **число** (цену).

Теперь другой сценарий: банк хочет решить, выдавать ли кредит клиенту. Ответ не число, а **категория**: «выдать» или «отказать». Это **классификация** — распределить объект по заранее известным группам.

Ключевой вопрос для разграничения: «Что мы предсказываем?»

- Если это **число на числовой шкале** (температура, вес, цена, расстояние) — **регрессия**
- Если это **метка из конечного набора** (спам/не спам, кот/собака/птица) — **классификация**

Техническое объяснение

Регрессия (regression) — задача обучения с учителем (supervised learning), в которой требуется построить модель $f: X \rightarrow R$, отображающую пространство признаков X в вещественнозначную непрерывную целевую переменную y .

Классификация (classification) — задача обучения с учителем, в которой целевая переменная y принимает значения из конечного множества классов: $y \in \{C_1, C_2, \dots, C_k\}$. При $k=2$ — бинарная классификация.

Подробные различия

Критерий	Регрессия	Классификация
Тип выхода	$y \in \mathbb{R}$ (непрерывное)	$y \in \{C_1, \dots, C_k\}$ (дискретное)
Функция потерь	MSE, MAE	Cross-entropy, Hinge loss
Метрики	RMSE, MAE, R^2	Accuracy, Precision, Recall, F1, AUC
Выход модели	Число	Класс или вектор вероятностей $P(y=C_k X)$
Примеры	Цена акции, температура, расход топлива	Спам-фильтр, диагноз, распознавание цифр

Границы размыты: задачу предсказания вероятности события (0.0–1.0) формально можно рассматривать как регрессию, но логистическая регрессия — классификационная модель. Ordinal regression — промежуточный случай: классы упорядочены (рейтинг 1–5).

Вопрос 2. Цель линейной регрессии. Математическая модель

Простое объяснение

Представь облако точек на графике: по оси X — площадь квартиры, по оси Y — цена. Задача линейной регрессии — провести **прямую линию** через это облако так, чтобы суммарное расстояние от всех точек до прямой было минимальным.

Когда признаков много (площадь И этаж И район), прямая превращается в «плоскость» в многомерном пространстве — **гиперплоскость**. Но математическая идея та же: найти уравнение этой плоскости, которое минимизирует ошибки предсказания.

Слово «линейная» означает не то, что данные расположены на прямой, а то, что модель **линейна относительно параметров β** . Это позволяет получить точное аналитическое решение — в отличие от нейросетей, которые требуют итеративного обучения.

Простая линейная регрессия (1 признак)

$$\hat{y}_i = \beta_0 + \beta_1 \cdot x_i + \varepsilon$$

Обозначения и интерпретация:

- $\hat{y}_i$ (y hat) — предсказанное значение для i -го наблюдения. Крышка указывает, что это оценка, а не истинное значение.
- β_0 (beta zero) — свободный член, intercept, смещение. Значение $\hat{y}$ когда $x = 0$. Например: базовая цена квартиры нулевой площади (физически бессмысленно, но математически необходимо).
- β_1 (beta one) — наклон прямой (slope). Показывает: если x вырастет на 1 единицу, $\hat{y}$ изменится на β_1 единиц. Например: каждый дополнительный кв.м добавляет β_1 тыс. руб. к цене.
- ε (epsilon) — случайная ошибка (шум). Всё, что модель не может объяснить: случайные рыночные колебания, неучтённые факторы. Предполагается $\varepsilon \sim N(0, \sigma^2)$.

Множественная линейная регрессия (p признаков)

$$\hat{y}_i = \beta_0 + \beta_1 \cdot x_{i1} + \beta_2 \cdot x_{i2} + \dots + \beta_p \cdot x_{ip}$$

В компактной матричной форме:

$$\hat{\mathbf{y}} = \mathbf{X}\boldsymbol{\beta}$$

Структура матриц:

- $\mathbf{X}$ — матрица признаков размером $(n \times (p+1))$, где первый столбец из единиц (для β_0)
- $\boldsymbol{\beta}$ — вектор весов $((p+1) \times 1)$
- $\hat{\mathbf{y}}$ — вектор предсказаний $(n \times 1)$

Пример для квартиры: $n=1000$ квартир, $p=3$ признака (площадь, этаж, расстояние до метро):

- $X[i] = [1, \text{площадь}_i, \text{этаж}_i, \text{км_до_метро}_i]$
- $\boldsymbol{\beta} = [\beta_0, \beta_1, \beta_2, \beta_3]^T$ — параметры, которые нужно найти
- $\hat{y}_i = \beta_0 + \beta_1 \cdot \text{площадь} + \beta_2 \cdot \text{этаж} + \beta_3 \cdot \text{км_до_метро}$

Линейность модели — по параметрам $\boldsymbol{\beta}$, а не по x . Это значит, что x^2 и $x \cdot z$ (взаимодействие) тоже допустимы как признаки — модель всё равно остаётся линейной по $\boldsymbol{\beta}$. Именно это используется в полиномиальной регрессии.

Вопрос 3. Методы оценки параметров. МНК

Простое объяснение

Нам нужно найти такие значения $\beta_0, \beta_1, \dots, \beta_p$, чтобы прямая (гиперплоскость) лежала как можно ближе ко всем точкам данных. «Близость» измеряется **суммой квадратов** вертикальных расстояний от каждой точки до линии.

Почему квадрат, а не просто расстояние? Во-первых, квадрат делает положительные и отрицательные ошибки одинаково «плохими» и не даёт им компенсировать друг друга. Во-вторых, квадратичная функция дифференцируема всюду — это ключевое для нахождения точного аналитического минимума.

Метод наименьших квадратов (МНК, Ordinary Least Squares — OLS) — находит такие β , при которых сумма квадратов остатков минимальна.

Функция потерь MSE

$$J(\beta) = (1/n) \cdot \sum_{i=1}^n (y_i - \hat{y}_i)^2 = (1/n) \cdot \|y - X\beta\|^2$$

Эту функцию нужно минимизировать по β . Берём частные производные и приравниваем к нулю:

$$\partial J / \partial \beta = -(2/n) \cdot X^T (y - X\beta) = 0$$

Аналитическое решение — нормальное уравнение (Normal Equation)

$$X^T X \beta = X^T y$$

$$\hat{\beta} = (X^T X)^{-1} \cdot X^T y$$

Это **точное замкнутое решение**. Никаких итераций — одна формула даёт оптимальные β .

Плюсы:

- Точное решение, нет гиперпараметров (learning rate), нет проблемы сходимости

Минусы:

- Инверсия матрицы $(X^T X)^{-1}$ — $O(p^3)$ операций. При $p = 10\,000$ признаков $\rightarrow 10^{12}$ операций, неприемлемо
- Требуется, чтобы $X^T X$ была невырожденной ($\det \neq 0$). Нарушается при мультиколлинеарности или $p > n$

Градиентный спуск (Gradient Descent)

Когда аналитическое решение слишком дорого, используют итерационный алгоритм.

Идея: представь, что стоишь на склоне горы в тумане. Чтобы спуститься в долину (минимум J), делай маленькие шаги в направлении самого крутого спуска.

$$\beta_j := \beta_j - \alpha \cdot \partial J / \partial \beta_j$$

Для MSE частная производная равна:

$$\partial J / \partial \beta_j = -(2/n) \cdot \sum_{i=1}^n (y_i - \hat{y}_i) \cdot x_{ij}$$

- α — learning rate (скорость обучения): слишком большой $\rightarrow$ не сходится, слишком маленький $\rightarrow$ медленно
- **Batch GD:** вся выборка на каждом шаге. Медленно, но точно
- **Stochastic GD (SGD):** одно наблюдение на шаге. Быстро, но шумно
- **Mini-batch GD:** батч из k наблюдений. Компромисс, стандарт в deep learning

Для линейной регрессии функция $J(\beta)$ — квадратичная, **выпуклая (convex)**. У неё ровно один глобальный минимум. Локальных минимумов нет — градиентный спуск всегда найдёт оптимальное решение.

Вопрос 4. Ошибка (остаток) в регрессии

Простое объяснение

После обучения модели для каждой точки данных у нас есть реальное значение y_i и предсказанное $\hat{y}_i$. Разница между ними — это **остаток (residual)**. Остатки — это то, чего модель не смогла объяснить.

Анализ остатков — это как «рентген» модели. Если остатки ведут себя случайно — модель хорошо уловила закономерность. Если в остатках есть паттерны — модель чего-то не учла.

Формула остатка

$$e_i = y_i - \hat{y}_i = y_i - (\beta_0 + \beta_1 \cdot x_i)$$

В матричной форме вектор остатков:

$$e = y - X\hat{\beta} = y - \hat{y}$$

Допущения Гаусса–Маркова

Теорема Гаусса–Маркова: при выполнении условий ниже, OLS-оценка $\hat{\beta}$ является **BLUE** (Best Linear Unbiased Estimator) — наилучшей линейной несмещённой оценкой.

- 1. **Линейность:** зависимость y от X действительно линейная. Если нет — модель систематически ошибается.
- 2. **Нулевое среднее ошибок:** $E[e_i] = 0$. Ошибки не имеют систематического смещения.
- 3. **Гомоскедастичность:** $Var(e_i) = \sigma^2 = \text{const}$ для всех i . Дисперсия ошибок одинакова по всей области. Нарушение — гетероскедастичность.
- 4. **Независимость:** $Cov(e_i, e_j) = 0$ при $i \neq j$. Ошибки не коррелируют между собой. Нарушается в временных рядах (автокорреляция).
- 5. **Нормальность:** $e_i \sim N(0, \sigma^2)$. Нужна для корректных доверительных интервалов и t -тестов.
- 6. **Отсутствие мультиколлинеарности:** признаки не являются линейными комбинациями друг друга.

Диагностика остатков

Метод	Что проверяет	Как должно выглядеть
Residual plot (e vs $\hat{y}$)	Линейность, гомоскедастичность	Случайное облако около нуля
QQ-plot	Нормальность остатков	Точки на диагональной прямой
Scale-Location plot	Гомоскедастичность	Горизонтальная полоса
Тест Бройша-Пагана	Гетероскедастичность (формально)	$p > 0.05$ = гомоскедастичность
Тест Дарбина-Уотсона	Автокорреляция остатков	Значение ≈ 2

Вопрос 5. Переобучение и регуляризация

Простое объяснение

Переобучение — это как студент, который перед экзаменом вызубрил все задачи из учебника дословно. На знакомых задачах он ответит идеально, но чуть измени условие — и он потеряется. Модель «запомнила» тренировочные данные, включая шум и случайности, и утратила способность обобщать.

Противоположная проблема — **недообучение (underfitting)**: модель слишком простая, не уловила никаких закономерностей. Как студент, который вообще не учился.

Регуляризация — это «штраф за зубрёжку». Мы говорим модели: «Ты можешь хорошо объяснять данные, но не позволяй коэффициентам становиться слишком большими». Большие коэффициенты — признак того, что модель подстраивается под шум.

Bias-Variance Tradeoff (компромисс смещения и дисперсии)

Ожидаемая ошибка модели разлагается на три компоненты:

$$E[(y - \hat{y})^2] = \text{Bias}^2 + \text{Variance} + \sigma^2 \text{ (шум)}$$

- **Bias (смещение)**: систематическая ошибка модели. Недообученная модель имеет высокий Bias — она не может уловить реальную зависимость. Пример: пытаться описать квадратичную зависимость прямой линией.
- **Variance (дисперсия)**: чувствительность модели к случайностям в тренировочных данных. Переобученная модель имеет высокую Variance — чуть изменишь данные, и предсказания кардинально изменятся.
- **σ^2 (irreducible noise)**: неустранимый шум данных. Даже идеальная модель не может его победить.
- **Цель**: найти баланс. Регуляризация немного увеличивает Bias, но сильно уменьшает Variance → итоговая ошибка на тесте уменьшается.

Общая форма с регуляризацией

$$J(\beta) = (1/n) \cdot \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \cdot \Omega(\beta)$$

λ (lambda) — гиперпараметр регуляризации, подбирается через кросс-валидацию:

- $\lambda = 0$: нет регуляризации, чистый OLS

- $\lambda \rightarrow \infty$: сверхсильный штраф, все коэффициенты $\rightarrow 0$ (модель предсказывает только среднее)
- Оптимальный λ : баланс между точностью на train и обобщением

Признаки переобучения

- Train error значительно меньше Test error (большой gap)
- Очень большие абсолютные значения коэффициентов β
- Высокая дисперсия при кросс-валидации (метрика сильно скачет по фолдам)
- Модель высокой степени (degree=10) идеально проходит через все точки, но нереалистична

Переобучение — главный враг ML на практике. Всегда проверяй модель на hold-out test set, который не использовался ни для обучения, ни для подбора гиперпараметров.

Вопросы 6–7. L1 vs L2 регуляризация. Когда применять

Простое объяснение

Обе регуляризации штрафуют за большие коэффициенты, но по-разному:

L2 (Ridge) — как резиновая лента, которая тянет все коэффициенты к нулю, но никогда не позволяет достичь нуля точно. Все признаки остаются в модели, просто с меньшим влиянием.

L1 (Lasso) — как ножницы, которые срезают слабые коэффициенты до точного нуля. Это встроенный отбор признаков: модель сама решает, какие признаки нерелевантны, и выбрасывает их из уравнения.

Геометрически: L2 — ограничение в форме сферы, L1 — в форме ромба. Углы ромба (на осях координат) соответствуют обнулению коэффициентов — именно там уровни функции потерь касаются границы ограничения.

L2 — Ridge Regression

$$J(\beta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \cdot \sum_{j=1}^p \beta_j^2$$

Штрафной член — L2-норма вектора β :

$$\Omega(\beta) = \|\beta\|^2 = \sum_{j=1}^p \beta_j^2$$

Аналитическое решение Ridge:

$$\hat{\beta}_{\text{ridge}} = (X^T X + \lambda I)^{-1} \cdot X^T y$$

Добавление λI к матрице делает её невырожденной — **Ridge решает проблему мультиколлинеарности!**

Свойства:

- Все коэффициенты уменьшаются пропорционально, ни один не обнуляется точно
- Хорошо работает когда все признаки вносят вклад, просто надо уменьшить их влияние
- Решает проблему мультиколлинеарности: когда два признака сильно коррелируют, Ridge делит вес между ними
- Аналитическое решение существует — быстро вычисляется

Когда применять: высокая мультиколлинеарность, все признаки потенциально важны

L1 — Lasso Regression (Least Absolute Shrinkage and Selection Operator)

$$J(\beta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \cdot \sum_{j=1}^p |\beta_j|$$

Штрафной член — L1-норма:

$$\Omega(\beta) = \|\beta\|_1 = \sum_{j=1}^p |\beta_j|$$

Свойства:

- Создаёт **sparse** (разреженные) решения: часть $\beta_j = 0$ точно
- Автоматический feature selection: нерелевантные признаки «отключаются»
- Нет аналитического решения (функция недифференцируема в 0), используется coordinate descent
- При очень большом λ большинство коэффициентов обнуляется

Когда применять: много признаков ($p \gg n$), предполагается что лишь малая часть важна (sparse signal). Пример: геномика — тысячи генов, только десятки влияют на болезнь.

Elastic Net — комбинация L1 и L2

$$J(\beta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda_1 \cdot \|\beta\|_1 + \lambda_2 \cdot \|\beta\|^2$$

- $l1_ratio = \lambda_1 / (\lambda_1 + \lambda_2) : 0 \rightarrow$ чистый Ridge, $1 \rightarrow$ чистый Lasso
- Решает проблему Lasso: при коррелирующих признаках Lasso случайно выбирает один, Elastic Net сохраняет группы

Сводная таблица

Критерий	Ridge (L2)	Lasso (L1)	Elastic Net
Штраф	$\sum \beta_j^2$	$\sum \beta_j $	$\lambda_1 \sum \beta_j + \lambda_2 \sum \beta_j^2$
Обнуляет коэффициенты	Нет	Да	Да
Отбор признаков	Нет	Да	Да
Аналитическое решение	Да	Нет	Нет
Мультиколлинеарность	Решает	Случайный выбор одного	Решает
Когда применять	Все признаки важны	Мало важных среди многих	Коррелирующие + отбор

Правило выбора: Ridge — если все признаки важны. Lasso — если нужен отбор признаков. Elastic Net — если признаки коррелируют и нужен отбор одновременно.

Вопросы 8–9. Полиномиальная регрессия. Нормализация

Простое объяснение

Линейная регрессия предполагает, что зависимость y от x — прямая линия. Но что если цена квартиры растёт с площадью нелинейно? Или зависимость температуры от времени суток — дуга, а не прямая?

Полиномиальная регрессия решает это элегантно: вместо одного признака x создаём несколько — $x, x^2, x^3, \dots$ — и обучаем обычную линейную регрессию на этих расширенных признаках. Кривая в пространстве x превращается в прямую в пространстве $(x, x^2, x^3, \dots)$.

Это как приготовить сложное блюдо из простых ингредиентов: сами ингредиенты (x, x^2, x^3) нелинейны, но «рецепт» (модель) по-прежнему линейный — $\beta_0 + \beta_1 \cdot x + \beta_2 \cdot x^2 + \dots$

Математическая модель

$$\hat{y}_i = \beta_0 + \beta_1 \cdot x_i + \beta_2 \cdot x_i^2 + \beta_3 \cdot x_i^3 + \dots + \beta_d \cdot x_i^d$$

Это линейная регрессия в пространстве $\phi(x_i) = [1, x_i, x_i^2, \dots, x_i^d]$. Все методы OLS применяются без изменений.

Степень	Форма кривой	Применение
d=1	Прямая линия	Обычная линейная регрессия
d=2	Парабола	U-образные зависимости
d=3	Кубическая кривая	S-образные зависимости
d > 5–7	Извилистая кривая	Опасная зона переобучения

Почему нужна нормализация

При возведении в степень значения признаков стремительно растут:

$$\begin{aligned} x = 100 &\rightarrow x^2 = 10^4 \rightarrow x^3 = 10^6 \rightarrow x^4 = 10^8 \\ x = 1000 &\rightarrow x^2 = 10^6 \rightarrow x^3 = 10^9 \rightarrow x^4 = 10^{12} \end{aligned}$$

Это вызывает серьёзные проблемы:

- **Численная нестабильность:** числа с плавающей запятой теряют точность
- **Матрица $X^T X$ становится плохо обусловленной** (ill-conditioned), инверсия неточна
- **Градиентный спуск сходится крайне медленно** из-за разных масштабов градиентов
- **Регуляризация работает некорректно:** штрафует признаки непропорционально масштабу

Min-Max Normalization

$$x_{\text{norm}} = (x - x_{\min}) / (x_{\max} - x_{\min}) \in [0, 1]$$

После нормализации $x \in [0, 1]$, тогда $x^2 \in [0, 1]$, $x^3 \in [0, 1]$ — масштаб не взрывается. Чувствительна к выбросам.

StandardScaler (Z-score)

$$x_{\text{std}} = (x - \mu) / \sigma$$

Результат: среднее=0, std=1. Рекомендуется для полиномиальной регрессии с Ridge/Lasso. Устойчивее к выбросам.

Вопрос 10. Метрики качества регрессии: RMSE, MAE, R²

Простое объяснение

После обучения модели нужно понять: насколько она хороша? Метрики — это «линейки» для измерения качества. Разные метрики расставляют акценты по-разному: одни строже относятся к большим ошибкам, другие — к маленьким.

Представь: модель предсказывает зарплату.

- **MAE** скажет: «В среднем ошибаемся на 15 000 руб»
- **RMSE** скажет: «Ошибаемся примерно на 20 000 руб, но учти — несколько сильных промахов подняли это число»
- **R²** скажет: «Модель объясняет 85% разброса зарплат»

MAE — Mean Absolute Error

$$\text{MAE} = (1/n) \cdot \sum_{i=1}^n |y_i - \hat{y}_i|$$

- Среднее абсолютное значение ошибок. **В тех же единицах, что у.**
- Интерпретация: «В среднем модель ошибается на MAE единиц»
- Устойчив к выбросам — линейный штраф, большие ошибки не доминируют
- Недифференцируем в точке $e_i = 0$ (угол)
- Хорошо подходит когда выбросы есть и они не должны сильно влиять на оценку

MSE и RMSE — Mean Squared Error / Root Mean Squared Error

$$\text{MSE} = (1/n) \cdot \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{[(1/n) \cdot \sum_{i=1}^n (y_i - \hat{y}_i)^2]}$$

- **MSE** — средний квадрат ошибки. Единицы y^2 . Дифференцируема — удобна для оптимизации
- **RMSE** — корень из MSE. Единицы y — удобнее для интерпретации
- Квадрат **штрафует большие ошибки значительно сильнее**: ошибка 10 → штраф 100, ошибка 100 → штраф 10 000

- Чувствителен к выбросам
- Хорошо для задач, где большие ошибки критичны (медицина, инженерия)

Сравнение на примере: ошибки [1, 1, 1, 1, 100] :

- $MAE = (1+1+1+1+100)/5 = 20.8$
- $RMSE = \sqrt{((1+1+1+1+10000)/5)} = \sqrt{2000.8} \approx 44.7$
- Видно: RMSE намного выше из-за одного выброса 100

R² — Коэффициент детерминации

$$R^2 = 1 - SS_{\text{res}} / SS_{\text{tot}} = 1 - \sum_{i=1}^n (y_i - \hat{y}_i)^2 / \sum_{i=1}^n (y_i - \bar{y})^2$$

Компоненты:

- **SS_res** (Sum of Squares Residual) = $\sum (y_i - \hat{y}_i)^2$ — необъяснённая дисперсия (ошибки модели)
- **SS_tot** (Sum of Squares Total) = $\sum (y_i - \bar{y})^2$ — общая дисперсия y
- **SS_reg** = $SS_{\text{tot}} - SS_{\text{res}}$ — объяснённая дисперсия (вклад модели)

Интерпретация R²:

Значение	Интерпретация
$R^2 = 1$	Идеальная модель, все ошибки = 0
$R^2 = 0.85$	Модель объясняет 85% дисперсии y
$R^2 = 0$	Модель не лучше предсказания среднего $\bar{y}$
$R^2 < 0$	Модель хуже простого предсказания среднего

Adjusted R² — скорректированный коэффициент

$$\bar{R}^2 = 1 - [(1 - R^2)(n - 1)] / (n - p - 1)$$

- n — число наблюдений, p — число признаков
- Штрафует за добавление нерелевантных признаков: R² может только расти при добавлении признаков (даже случайных), Adjusted R² может снижаться

MAPE — Mean Absolute Percentage Error

$$MAPE = (100\%/n) \cdot \sum_{i=1}^n |(y_i - \hat{y}_i) / y_i|$$

- Показывает относительную ошибку в процентах — легко интерпретировать бизнесу
- Проблема: не определена при $y_i = 0$; асимметрична

Какую метрику выбрать? Для бизнеса — MAE или MAPE (интерпретируемы). Для оптимизации — MSE (дифференцируема). Для отчётности — R^2 . При выбросах — MAE надёжнее RMSE.

Вопрос 11. Почему корреляция — ненадёжный показатель качества модели

Простое объяснение

Корреляция между признаком и целевой переменной — это показатель **линейной связи** между ними. Высокая корреляция ($r = 0.9$) между площадью и ценой говорит: «Они движутся вместе». Но это не то же самое, что хорошая модель!

Классический пример — квартет Анскомба (1973): четыре совершенно разных датасета, у каждого $r \approx 0.816$ и $R^2 \approx 0.67$. Один — действительно линейный, второй — квадратичный, третий — линейный с одним выбросом, четвёртый — вертикальный кластер с одной точкой-выбросом.

Формула коэффициента корреляции Пирсона

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{(\sum_{i=1}^n (x_i - \bar{x})^2)} \cdot \sqrt{(\sum_{i=1}^n (y_i - \bar{y})^2)}}$$

- $r \in [-1, 1]$: $r=1$ — идеальная прямая зависимость, $r=-1$ — обратная, $r=0$ — нет линейной связи
- Важно: $r^2 = R^2$ **только** в простой линейной регрессии (1 признак)

Почему корреляция ненадёжна

1. **Только линейная связь:** корреляция Пирсона не уловит зависимость $y = x^2$. Например: $r(x, x^2) = 0$ если x симметрично распределён, но зависимость очевидна.
2. **Квартет Анскомба:** $r \approx 0.816$ у четырёх принципиально разных датасетов.
3. **Ложные корреляции (spurious correlations):** потребление мороженого коррелирует с утоплениями (обе переменные растут летом). Это не значит «мороженое убивает».
4. **Мультиколлинеарность:** если два признака сильно коррелируют друг с другом, OLS нестабилен — малое изменение данных резко меняет коэффициенты.

5. **$r(X,y) \neq$ качество модели:** признак с $r = 0.5$ может быть бесполезен, если другой признак с $r = 0.7$ уже в модели и они коррелируют.
6. **Чувствительность к выбросам:** один нетипичный объект может резко поднять или опустить r .

Вместо корреляции используйте: R^2 / Adjusted R^2 для оценки модели, VIF (Variance Inflation Factor) для диагностики мультиколлинеарности, Partial correlations для оценки вклада признака при контроле остальных.

Вопрос 12. Кросс-валидация и зачем она нужна

Простое объяснение

Представь, что оцениваешь уровень игрока в шахматы. Если провести всего одну партию — оценка ненадёжна (мог повезти или не повезти). Лучше сыграть 10 партий с разными соперниками и усреднить результат. Это принцип **кросс-валидации**.

Проблема с одним train/test разбиением: если «удачно» разбили данные, модель выглядит лучше, чем есть. Кросс-валидация делает оценку **независимой от конкретного разбиения** — мы проверяем модель на всех частях данных по очереди.

k-Fold Cross-Validation (k-кратная кросс-валидация)

Алгоритм:

1. Разделить данные на k равных частей (фолдов): $D_1, D_2, \dots, D_k$
2. Для $i = 1, 2, \dots, k$:
 - Обучить модель на всех фолдах, кроме D_i
 - Оценить модель на D_i
 - Записать $Score_i$
3. Итоговый результат: среднее и стандартное отклонение по k оценкам

$$CV_Score = (1/k) \cdot \sum_{i=1}^k Score_i$$

$$CV_Std = \sqrt{[(1/(k-1)) \cdot \sum_{i=1}^k (Score_i - CV_Score)^2]}$$

- **$k = 5$ или $k = 10$** — стандартные значения
- Каждое наблюдение попадает в validation **ровно один раз**

Leave-One-Out Cross-Validation (LOO-CV)

Крайний случай: $k = n$. Каждый раз модель обучается на $n-1$ наблюдениях, тестируется на одном.

- Максимальная точность оценки, но требует n обучений — очень медленно при больших n
- Для линейных моделей LOO имеет эффективную формулу без n отдельных обучений

Зачем нужна кросс-валидация

- **Оценка обобщающей способности:** насколько модель хороша на новых данных
- **Подбор гиперпараметров:** GridSearchCV перебирает комбинации (λ , degree) и выбирает лучшую по CV-score
- **Сравнение моделей:** какая модель стабильнее? CV_Std показывает дисперсию оценки
- **Малые данные:** вместо выделения отдельного val set, CV использует все данные эффективно

```
from sklearn.model_selection import cross_val_score
cross_val_score(model, X, y, cv=5, scoring='neg_mean_squared_error')
```

Вопрос 13. Когда линейная регрессия, когда нелинейные методы

Линейная регрессия — применять когда

- Зависимость y от X **линейна или близка к ней** (проверяй через EDA и residual plots)
- Важна **интерпретируемость**: нужно объяснить результат клиенту. «Каждый год опыта добавляет 5000 руб. к зарплате»
- **Мало данных** ($n < 1000$): нелинейные методы переобучаются на малых выборках
- Нужен **быстрый baseline**: всегда начинай с линейной — если нелинейная модель не лучше на 5–10%, усложнение не оправдано
- **Высокая размерность** ($p \gg n$): линейная + Lasso хорошо работает
- **Регуляторная среда**: банки, медицина — требуют объяснимых моделей

Нелинейные методы — применять когда

- **Явная нелинейность**: квадратичные, экспоненциальные, пороговые зависимости
- **Сложные взаимодействия**: цена квартиры зависит от площадь $\times$ район (нелинейно)
- **Много данных** ($n > 10\,000$): нелинейные методы раскрываются при большом объёме

- **Разнотипные признаки:** числа + категории + тексты — деревья обрабатывают естественно
- Высокая предсказательная точность важнее интерпретируемости

Популярные нелинейные методы для регрессии

Метод	Особенности
Decision Tree	Интерпретируемо, но переобучается без pruning
Random Forest	Ансамбль деревьев, устойчив к выбросам и шуму
Gradient Boosting (XGBoost, LightGBM, CatBoost)	Топ для табличных данных
SVR (Support Vector Regression)	Хорош при небольших выборках
Neural Networks	Для изображений, текстов, сложных зависимостей

Практическое правило: всегда начинай с LinearRegression как baseline → Ridge/Lasso → Random Forest → Gradient Boosting.

Вопрос 14. Интерпретация важности признаков в линейной регрессии

Простое объяснение

Каждый признак в линейной регрессии имеет коэффициент β . Знак коэффициента говорит о направлении влияния (+/-), абсолютное значение — о силе. Но сравнивать коэффициенты напрямую **нельзя**, если признаки в разных единицах измерения!

Пример: $\beta(\text{площадь в кв.м}) = 50$, $\beta(\text{расстояние до метро в км}) = -300$. Значит ли это, что расстояние важнее? **Нет!** Нужно учесть масштаб: площадь варьируется от 20 до 200 (+180 кв.м), расстояние — от 0 до 5 (+5 км). После стандартизации сравнение станет корректным.

Стандартизированные коэффициенты (Beta coefficients)

После применения StandardScaler к признакам:

$$\beta_{std,j} = \beta_j \cdot (\sigma_j / \sigma_y)$$

- $|\beta_{std,j}|$ — прямой показатель важности признака j
- Все признаки в одних единицах (стандартных отклонениях) — корректно сравнивать
- $\beta_{std} = 0.3$ означает: при росте признака j на 1σ , y растёт на 0.3σ

Permutation Feature Importance

Идея: если признак важен, то после случайного перемешивания его значений метрика должна сильно ухудшиться.

1. Оцени базовую метрику модели на test set: $Score_{base}$
2. Для каждого признака j : перемешай столбец $j \rightarrow$ вычисли $Score_j$
3. Важность $j = Score_{base} - Score_j$ (чем больше — тем важнее)

$$Importance_j = Score_{base} - Score_{j, permuted}$$

- Работает для **любой модели**, не только линейной
- Минус: медленно, нестабильно для коррелирующих признаков

SHAP — SHapley Additive exPlanations

Продвинутый метод, основанный на теории игр (кооперативные игры Шепли). Для каждого наблюдения вычисляет вклад каждого признака в предсказание:

$$\hat{y}_i = \phi_0 + \sum_{j=1}^p \phi_j$$

- ϕ_0 — базовое значение (среднее предсказание модели)
- ϕ_j — вклад признака j в конкретное предсказание (может быть + и −)

p-values коэффициентов (statsmodels)

- $H_0: \beta_j = 0$ (признак j не влияет на y)
- $p < 0.05 \rightarrow$ коэффициент статистически значим
- t-статистика: $t = \hat{\beta}_j / SE(\hat{\beta}_j)$

```
import statsmodels.api as sm
model = sm.OLS(y, sm.add_constant(X)).fit()
print(model.summary())
```

Вопрос 15. Библиотеки Python для линейной и полиномиальной регрессии

scikit-learn (sklearn) — основная библиотека ML

Класс/функция	Назначение
<code>LinearRegression</code>	Базовая линейная регрессия (OLS). Методы <code>fit()</code> , <code>predict()</code> , атрибуты <code>coef_</code> , <code>intercept_</code>
<code>Ridge(alpha=1.0)</code>	L2-регуляризация. Параметр $\alpha = \lambda$
<code>Lasso(alpha=1.0)</code>	L1-регуляризация. Автоматический feature selection
<code>ElasticNet(alpha, l1_ratio)</code>	Комбинация L1+L2
<code>PolynomialFeatures(degree=2)</code>	Создание полиномиальных признаков x , x^2 , $x \cdot y$
<code>StandardScaler</code> , <code>MinMaxScaler</code>	Нормализация
<code>Pipeline</code>	Объединение preprocessing и модели
<code>GridSearchCV</code> , <code>cross_val_score</code>	Подбор гиперпараметров и кросс-валидация
<code>RidgeCV</code> , <code>LassoCV</code>	Встроенная кросс-валидация для подбора α

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV, cross_val_score,
train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score
```

numpy — числовые вычисления

- `np.linalg.lstsq(X, y)` : прямое решение МНК без scikit-learn
- `np.linalg.inv(A)` : инверсия матрицы для нормального уравнения
- `np.dot(A, B)` : матричное умножение

statsmodels — статистический анализ

- OLS: полная статистика модели — R^2 , F-statistic, p-values коэффициентов, AIC/BIC
- Доверительные интервалы, тесты на гетероскедастичность и нормальность остатков

pandas — работа с данными

- `pd.read_csv()`, `df.describe()`, `df.isnull().sum()`, `pd.get_dummies()`, `df.corr()`

matplotlib / seaborn — визуализация

- `sns.heatmap(df.corr())` : тепловая карта корреляций
- `plt.scatter(y_test, y_pred)` : предсказанные vs реальные
- `sns.residplot()` : диагностика остатков

Вопрос 16. Применение линейной регрессии к категориальным данным

Простое объяснение

Линейная регрессия работает исключительно с числами. Но в реальных данных часто есть текстовые категории: город, жанр, цвет, образование.

Главная ловушка: просто заменить Москва=1, Казань=2, Новосибирск=3 **нельзя!** Тогда модель решит, что Новосибирск «в 3 раза важнее» Москвы, или что Казань находится «между» ними.

Решение — One-Hot Encoding: каждая категория получает отдельный бинарный признак.

- Москва = [1, 0, 0]
- Казань = [0, 1, 0]
- Новосибирск = [0, 0, 1]

Dummy Variable Trap

Если для k категорий создать k бинарных столбцов, то их сумма всегда = 1:

$$is_Москва + is_Казань + is_Нск \equiv 1$$

Это идеальная мультиколлинеарность — матрица $X^T X$ становится сингулярной. **Решение:** убрать один столбец (базовую категорию). Тогда $k-1$ столбцов кодируют все k категорий.

Сравнение методов кодирования

Метод	Формула	Когда применять	Опасности
Label Encoding	Москва → 0, Казань → 1	Только ordinal данные	Ложный порядок для nominal
One-Hot Encoding	k категорий → k-1 столбцов	Nominal, < 20 категорий	Высокая размерность
Target Encoding	Категория → среднее y в группе	Высокая кардинальность	Data leakage без CV
Binary Encoding	Категория → бинарный код	Высокая кардинальность	Потеря информации

```
# pandas
pd.get_dummies(df, columns=['city'], drop_first=True)

# sklearn
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(drop='first', sparse_output=False)
X_enc = enc.fit_transform(X_train[cat_cols])
```

Правило: ONE для категорий с < 20 уникальными значениями. Target/Binary Encoding для высококардинальных признаков.

ЧАСТЬ II: ТЕОРИЯ ПРЕПРОЦЕССИНГА ДАННЫХ

Вопрос 1. Библиотеки для работы с датасетами

pandas — работа с табличными данными

Функция	Назначение
pd.read_csv(), pd.read_excel()	Загрузка данных
df.head(10), df.tail(5)	Просмотр первых/последних строк
df.info()	Типы столбцов, количество ненулевых значений
df.describe()	Статистики: mean, std, min, max, percentiles
df.isnull().sum()	Количество пропусков по каждому столбцу
df.value_counts()	Частоты уникальных значений

Функция	Назначение
<code>df.groupby()</code> , <code>df.merge()</code>	Агрегация и объединение
<code>pd.get_dummies()</code>	One-hot encoding

numpy — числовые вычисления

Функция	Назначение
<code>np.array()</code> , <code>np.zeros()</code> , <code>np.ones()</code>	Создание массивов
<code>np.nan</code> , <code>np.isnan()</code> , <code>np.nanmean()</code>	Работа с NaN
<code>np.percentile()</code> , <code>np.quantile()</code>	Вычисление перцентилей (для IQR)
<code>np.random.seed()</code>	Фиксация случайности для воспроизводимости
<code>np.linalg</code>	Матричная алгебра

scikit-learn preprocessing

Класс	Назначение
<code>StandardScaler</code>	Стандартизация ($\mu=0$, $\sigma=1$)
<code>MinMaxScaler</code>	Нормализация в [0,1]
<code>RobustScaler</code>	По медиане и IQR (устойчив к выбросам)
<code>SimpleImputer</code>	Заполнение пропусков (mean, median, most_frequent)
<code>KNNImputer</code>	Заполнение по k ближайшим соседям
<code>OneHotEncoder</code> , <code>OrdinalEncoder</code>	Кодирование категорий
<code>PolynomialFeatures</code>	Расширение признаков
<code>ColumnTransformer</code>	Разные трансформации для разных столбцов
<code>Pipeline</code>	Последовательная цепочка трансформаций + модель

scipy — статистические функции

- `scipy.stats.zscore()` : Z-score для обнаружения выбросов
- `scipy.stats.shapiro()` : тест Шапиро-Уилка на нормальность
- `scipy.stats.pearsonr()` , `spearmanr()` : корреляции с p-values

matplotlib / seaborn — визуализация

- `sns.histplot()`, `plt.hist()` : гистограммы распределений
 - `sns.boxplot()` : ящики с усами — наглядно показывают выбросы
 - `sns.heatmap(df.corr(), annot=True)` : тепловая карта корреляций
 - `sns.pairplot(df)` : матрица scatter plots для EDA
-

Вопрос 2. Что такое препроцессинг данных? Почему он важен

Простое объяснение

Реальные данные — как сырые овощи: перед готовкой нужно помыть, почистить, нарезать. ML-алгоритм — это рецепт. Если подать немытые, гнилые овощи (грязные данные) — блюдо (модель) будет несъедобным, сколько бы ни был хорош рецепт.

Препроцессинг — это всё, что делают с данными **до** обучения модели: заполняют пропуски, убирают аномалии, преобразуют типы, масштабируют значения, кодируют категории. В реальных проектах препроцессинг занимает **60–80% времени аналитика**.

Принцип GIGO (Garbage In — Garbage Out): «Мусор на входе — мусор на выходе».

Почему препроцессинг критически важен

1. **Математические требования алгоритмов:** большинство алгоритмов требуют числовых входных данных без NaN. Матрицу $X^T X$ нельзя вычислить, если в X есть бесконечности или NaN.
2. **Масштаб влияет на результат:** рост (0.5–2.1 м) и зарплата (10 000–5 000 000 руб) в одной модели: градиентный спуск будет «видеть» зарплату в миллион раз важнее. Регуляризация штрафует коэффициенты непропорционально.
3. **Пропуски искажают оценки:** sklearn по умолчанию выбрасывает строки с NaN, теряя ценные данные. Неправильное заполнение пропусков создаёт систематическое смещение.
4. **Выбросы нарушают OLS:** МНК минимизирует квадраты ошибок. Один выброс с ошибкой 100 имеет штраф 10 000 — в 10 000 раз больше, чем точка с ошибкой 1. Выброс «тянет» всю линию к себе.
5. **Обобщающая способность:** правильный препроцессинг (fit только на train) предотвращает data leakage — утечку информации из test в модель.
6. **Качество данных:** дублированные строки, опечатки в категориях ("Москва", "москва", "МОСКВА" — три разные категории!) — без исправления модель обучится неверно.

Вопрос 3. Основные этапы препроцессинга

Этап 1: EDA — Exploratory Data Analysis

- Изучение структуры: `df.info()` , `df.describe()` , `df.shape`
- Типы признаков: числовые, категориальные, дата/время, текст
- Распределения: гистограммы, box plots — есть ли перекося, выбросы?
- Пропуски: `df.isnull().sum()` — где и сколько?
- Корреляции: heatmap — какие признаки связаны?
- Целевая переменная: её распределение, есть ли дисбаланс?

Этап 2: Очистка данных

- Удаление дубликатов: `df.drop_duplicates()`
- Исправление типов: `pd.to_numeric()` , `pd.to_datetime()`
- Стандартизация категорий: `.str.lower()` , `.str.strip()`
- Обработка пропусков: заполнение или удаление
- Обработка выбросов: обнаружение и исправление

Этап 3: Feature Engineering

- Создание новых признаков: $\text{возраст} = \text{текущий год} - \text{год рождения}$
- Взаимодействия: $\text{площадь} \times \text{этаж}$, $\text{ценазакв_м} = \text{цена} / \text{площадь}$
- Трансформации: `log(цена)` , `sqrt(расстояние)` — для нормализации распределения
- Извлечение из дат: год, месяц, день_недели, квартал
- Бинаризация: `is_weekend = (день_недели >= 5)`

Этап 4: Кодирование категорий

- Nominal → OHE (One-Hot Encoding)
- Ordinal → Label/Ordinal Encoding с заданным порядком
- Высококардинальные → Target/Binary/Frequency Encoding

Этап 5: Разбиение на выборки

- `train_test_split()` до любой нормализации
- Стратификация при дисбалансе классов

Этап 6: Масштабирование (только на train!)

- `scaler.fit(X_train) → scaler.transform(X_train)` и `scaler.transform(X_test)`
- StandardScaler для большинства алгоритмов
- MinMaxScaler для нейронных сетей и изображений

Правильная последовательность критична! Масштабирование до разбиения = data leakage. Feature engineering лучше делать до разбиения (если не использует y), кодирование — после fit на train.

Вопрос 4. Методы заполнения пропусков

Классификация пропусков

Тип	Расшифровка	Пример
MCAR	Missing Completely At Random	Прибор случайно вышел из строя
MAR	Missing At Random (зависит от других переменных)	Пожилые реже указывают доход
MNAR	Missing Not At Random (зависит от самого значения)	Богатые не указывают доход

Методы работы с пропусками

1. Удаление:

- Listwise: удалить строку при любом пропуске. Безопасно только при MCAR и < 5% пропусков
- Удаление признака: если признак имеет > 50–70% пропусков

2. Простое заполнение (Simple Imputation):

- **Mean:** заменяем NaN средним. Только для нормально распределённых данных без выбросов
- **Median:** заменяем медианой. Устойчива к выбросам, рекомендуется как default
- **Mode:** для категориальных — самое частое значение
- **Constant:** заменяем фиксированным числом (0, -1, 'Unknown')

3. KNN Imputation: находим k ближайших соседей по остальным признакам, берём их среднее для пропущенного.

$$\text{imputed}(x)_{ij} = (1/k) \cdot \sum_{l=1}^k x_{lj}$$

4. MICE (Multiple Imputation by Chained Equations):

- Строится отдельная регрессионная модель для каждого признака с пропусками
- Наиболее статистически корректный метод для MAR данных
- sklearn: `IterativeImputer` (экспериментальный)

5. Indicator Variable (флаг пропуска): создаём бинарный признак `was_missing = 1` если значение было пропущено.

```
from sklearn.impute import SimpleImputer, KNNImputer
imputer = SimpleImputer(strategy='median')
X_train_imputed = imputer.fit_transform(X_train)
X_test_imputed = imputer.transform(X_test) # те же параметры!
```

Как и со scaler: `imputer.fit()` только на train! Иначе медиана test будет «просочена» в модель.

Вопрос 5. Нормализация и стандартизация. Когда применять

Простое объяснение

Проблема: признаки имеют разные масштабы. Рост человека: 1.6–1.9 м. Зарплата: 30 000–3 000 000 руб. Если подать всё это «как есть» в алгоритм, он будет считать зарплату намного важнее, просто потому что её числа большие.

Когда scaling ОБЯЗАТЕЛЕН

Алгоритмы, основанные на расстояниях (KNN, SVM), градиентный спуск (линейная регрессия, нейросети), алгоритмы с регуляризацией (Ridge, Lasso), PCA.

Когда scaling НЕ НУЖЕН

Деревья решений и их ансамбли (Random Forest, Gradient Boosting) — они делят по пороговым значениям, масштаб не важен. Наивный Байес.

Методы масштабирования

Min-Max Normalization (MinMaxScaler):

$$x_{\text{norm}} = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}}) \in [0, 1]$$

- Результат строго в $[0, 1]$
- Минус: чувствительна к выбросам
- Применять: нейросети, изображения

Z-score Standardization (StandardScaler):

$$x_{\text{std}} = (x - \mu) / \sigma$$

- Результат: среднее = 0, std = 1. Не ограничен диапазоном
- Устойчивее к выбросам
- Применять: Ridge/Lasso, PCA, SVM, логистическая регрессия

RobustScaler — для данных с выбросами:

$$x_{\text{robust}} = (x - \text{median}(x)) / \text{IQR}$$

- Использует медиану и IQR вместо среднего и σ

Сводная таблица

Метод	Когда применять	Устойчивость к выбросам
StandardScaler	Default для большинства	Средняя
MinMaxScaler	Нейросети, изображения, нужен $[0,1]$	Низкая
RobustScaler	Данные с выбросами	Высокая
Log/sqrt	Сильно перекошенные распределения	—

Вопрос 6. Выбросы: обнаружение и удаление

Простое объяснение

Выброс (outlier) — наблюдение, сильно отличающееся от остальных. Рост человека 4 метра, возраст −5 лет — это ошибки. Но выброс может быть и реальным: Илон Маск с состоянием \$200 млрд в датасете «богатых людей» — это не ошибка.

Для OLS: квадратичный штраф → выбросы доминируют в функции потерь. Один выброс может «перетянуть» всю линию регрессии.

Метод IQR (Tukey's Fences)

```
IQR = Q3 - Q1
Нижняя граница: Q1 - 1.5 * IQR
Верхняя граница: Q3 + 1.5 * IQR
```

- Q1 — 25-й перцентиль, Q3 — 75-й перцентиль
- IQR охватывает «средние» 50% данных
- Коэффициент 1.5: стандарт Тьюки. При 3.0 — только «экстремальные» выбросы
- Визуально — box plot

Z-score метод

```
|z| = |(x - μ) / σ| > 3 → выброс
```

- При нормальном распределении 99.7% точек в ±3σ
- Проблема: μ и σ сами чувствительны к выбросам (парадокс маскировки)

Isolation Forest

- ML-алгоритм: строит случайные деревья, изолируя точки
- Аномалии изолируются быстрее (короткими путями в дереве)
- Работает в многомерном пространстве

Что делать с выбросами

Метод	Описание	Когда
Удаление	Убрать строку	Уверены, что это ошибка, < 1–5%
Winsorizing	Заменить граничными значениями	Не хотим терять строку
Log-трансформация	$y' = \log(1+y)$	Right-skewed данные
Устойчивые методы	HuberRegressor, RANSACRegressor	Замена OLS

Вопрос 7. Категориальные и числовые признаки. Преобразование

Классификация типов данных

Числовые (Quantitative):

- **Непрерывные (continuous):** бесконечно делимы. Рост, температура, цена.
Операции: +, -, ×, ÷
- **Дискретные (discrete):** счётное число значений. Количество комнат (1,2,3,4,...)

Категориальные (Qualitative):

- **Номинальные (nominal):** нет порядка. Цвет, город, порода собаки. Операции: только равенство/неравенство
- **Порядковые (ordinal):** есть порядок, но расстояния неопределены. Рейтинг 1–5, уровень образования

Методы преобразования

Label Encoding: Москва → 0, Казань → 1, Новосибирск → 2

- Только для ordinal данных с явным порядком
- Для nominal: модель считает $2 > 1 > 0$, т.е. Новосибирск «больше» Казани

One-Hot Encoding (OHE): для nominal

- k категорий → k-1 столбцов (drop_first=True)
- Проблема: при k=1000 → 999 новых столбцов

Target Encoding:

$$TE(c_k) = \frac{\sum(y_i, \text{ где } x_i = c_k)}{\text{count}(c_k)}$$

- Компактно, но опасность data leakage

Binary Encoding: k категорий → $\lceil \log_2(k) \rceil$ бинарных столбцов. Экономнее OHE.

Вопрос 8. Разбиение данных: Train / Validation / Test

Зачем три разных набора

- **Train set:** модель на нём обучается — оптимизирует β . Метрика на train — не показатель обобщения.
- **Validation set:** не участвует в обучении. Для подбора гиперпараметров.
- **Test set:** используется **ОДИН РАЗ** для финальной оценки.

Типичные пропорции

Размер датасета	Train	Validation	Test
< 1 000	60%	20%	20%
1 000 – 100 000	70–80%	10–15%	10–15%
> 1 000 000	90–98%	1–5%	1–5%

Параметры разбиения

- **random_state:** фиксирует случайность — разбиение воспроизводимо
- **stratify=y:** сохраняет пропорции классов
- **shuffle=True (default):** перемешать данные. False для временных рядов!

```
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3,
random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp,
test_size=0.5)
```

При малых данных используй **кросс-валидацию** вместо фиксированного val set.

Вопрос 9. Разные операции препроцессинга для разных признаков

Тип признака	Пропуски	Масштабирование	Кодирование
Числовые непрерывные	Медиана / среднее	StandardScaler / MinMaxScaler	—
Числовые дискретные	Медиана / мода / 0	StandardScaler	—
Категориальные nominal	Мода / 'Unknown'	Нет!	OHE (drop_first=True)

Тип признака	Пропуски	Масштабирование	Кодирование
Категориальные ordinal	Мода	Можно после OrdinalEncoder	OrdinalEncoder с порядком
Бинарные (0/1)	—	Не нормализуем	—
Дата/время	—	—	Извлечь компоненты (год, месяц)

Циклические признаки (месяц, час)

```
sin_month = sin(2π · month / 12)
cos_month = cos(2π · month / 12)
```

Январь (1) и Декабрь (12) становятся «близко» — sin/cos это обеспечивает.

ColumnTransformer

```
from sklearn.compose import ColumnTransformer
ct = ColumnTransformer([
    ('num', StandardScaler(), numerical_cols),
    ('cat', OneHotEncoder(drop='first'), categorical_cols),
    ('ord', OrdinalEncoder(categories=[...]), ordinal_cols)
])
X_transformed = ct.fit_transform(X_train)
```

Вопрос 10. Нормализация данных после разбиения Train/Test

Простое объяснение

Это одно из самых важных правил в ML: нормализовать данные нужно **ТОЛЬКО после** того, как разбил их на train и test. Если нормализовать до разбиения — это data leakage.

Конкретный пример: ты хочешь нормализовать зарплату. Считаешь μ и σ по **всему** датасету (включая test). Но тогда информация о test set (его статистики μ и σ) просочилась в обучение.

Data Leakage — утечка данных

Виды data leakage:

- **Target leakage:** в признаках есть информация о y , которой не будет при реальном предсказании
- **Train-test contamination:** статистики (μ , σ , min, max) вычислены по всем данным до разбиения
- **Feature engineering leakage:** target encoding вычислен до CV-fold разбиения
- **Temporal leakage:** обучение на будущих данных для предсказания прошлого

Правильный порядок нормализации

```
# Шаг 1: разбить данные
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Шаг 2: fit scaler ТОЛЬКО на train
scaler = StandardScaler()
scaler.fit(X_train) # вычисляет  $\mu$  и  $\sigma$  только по train!

# Шаг 3: transform и train, и test с ОДНИМИ параметрами
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test) # те же  $\mu$  и  $\sigma$  что у train
```

Pipeline — автоматическая защита от leakage

```
from sklearn.pipeline import Pipeline
pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('model', Ridge(alpha=1.0))
])
pipe.fit(X_train, y_train) # fit всего пайплайна на train
pipe.score(X_test, y_test) # transform применяется автоматически
cross_val_score(pipe, X, y, cv=5) # leakage невозможен!
```

Правило большого пальца: всё что считается по данным (μ , σ , медиана, частоты категорий) — fit только на train. Всё остальное (удаление дубликатов, исправление типов) — можно до разбиения.

ЧАСТЬ III: ПОДРОБНЫЙ РАЗБОР КОДА НОУТБУКА

Ячейка 1: Импорт библиотек и загрузка данных

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("calories_burned_data.csv")

print(df.shape)
print(df.head())
print(df.dtypes)
print(df.describe())
```

Построчный разбор

```
import pandas as pd
```

Импортируем библиотеку **pandas** — основной инструмент Python для работы с табличными данными. Псевдоним `pd` — общепринятое сокращение. Pandas предоставляет структуру `DataFrame` (таблица со строками и столбцами) и сотни методов для фильтрации, агрегации, преобразования данных.

```
import numpy as np
```

Импортируем **numpy** — библиотеку для работы с многомерными массивами и математическими операциями. Все вычисления в `scikit-learn` под капотом используют `numpy`-массивы. Нам понадобится для генерации случайных чисел (`np.random`), математических функций и работы с массивами.

```
import matplotlib.pyplot as plt
```

Импортируем модуль **pyplot** из библиотеки **matplotlib** — низкоуровневая библиотека визуализации. Через `plt` мы управляем фигурами, осями, подписями, заголовками. Каждый вызов `plt.figure()`, `plt.show()` и т.д. — это работа с этим модулем.

```
import seaborn as sns
```

Импортируем **seaborn** — надстройка над `matplotlib` с более красивыми и информативными графиками «из коробки». Предоставляет `heatmap`, `histplot`, `barplot` и другие визуализации, оптимизированные для статистического анализа.

```
df = pd.read_csv("calories_burned_data.csv")
```

Загружаем CSV-файл в `DataFrame`. `pd.read_csv()` принимает:

- **Первый аргумент** — путь к файлу (строка). Может быть относительным или абсолютным.

- Автоматически определяет разделитель (запятая по умолчанию), заголовки столбцов (первая строка по умолчанию), типы данных.
- Возвращает объект `DataFrame` — таблицу со строками (индексы) и столбцами (имена из заголовка файла).
- Результат сохраняется в переменную `df` (сокращение от `DataFrame` — общепринятое имя).

`print(df.shape)`

Атрибут `.shape` возвращает кортеж (`n_строк`, `n_столбцов`). Результат: `(200, 10)` — 200 наблюдений, 10 признаков (включая целевую переменную). Это первая проверка: достаточно ли данных для обучения? Слишком мало строк — модель будет ненадёжной.

`print(df.head())`

Метод `.head(n)` возвращает первые `n` строк (по умолчанию 5). Позволяет визуально оценить:

- Какие столбцы есть в данных
- Какие значения они принимают
- Есть ли очевидные проблемы (строки вместо чисел, NaN, нетипичные значения)

`print(df.dtypes)`

Атрибут `.dtypes` показывает тип данных каждого столбца. Важно проверить:

- Числовые столбцы должны иметь тип `int64` или `float64`
- Текстовые — `object` или `str`
- Если столбец с числами имеет тип `object` — значит где-то есть нечисловые значения (ошибки, пропуски, текст)
- В нашем случае: `Gender` имеет тип `str` — это категориальный признак, его нужно будет закодировать

`print(df.describe())`

Метод `.describe()` вычисляет сводную статистику по каждому числовому столбцу:

- **count** — количество непустых значений (если `count < shape[0]`, есть пропуски)
- **mean** — среднее значение
- **std** — стандартное отклонение (разброс данных)
- **min** — минимальное значение (проверка на аномалии: отрицательный возраст?)
- **25% (Q1)** — первый квартиль
- **50% (медиана)** — центральное значение
- **75% (Q3)** — третий квартиль
- **max** — максимальное значение

По разнице min/max и среднему можно быстро оценить, есть ли выбросы и как распределены данные.

Ячейка 2: Проверка на пропуски

```
print(df.isnull().sum())
```

Построчный разбор

`df.isnull()` — возвращает DataFrame той же формы, где каждое значение заменено на `True` (если NaN) или `False` (если не NaN). Это маска пропусков.

`.sum()` — суммирует `True` значения по каждому столбцу (`True = 1`, `False = 0`). Результат — количество пропусков в каждом столбце.

Зачем проверять пропуски:

- Большинство алгоритмов sklearn **не принимают NaN** на вход — выдадут ошибку
- Пропуски нужно либо заполнить (imputation), либо удалить строки/столбцы
- Количество пропусков определяет метод: `< 5%` — можно удалить строки, `> 50%` — удалить столбец, промежуточные — заполнить (median, KNN)

Результат: все столбцы содержат 0 пропусков — данные чистые, дополнительная обработка не требуется.

Ячейка 3: Проверка на дубликаты

```
print(df.duplicated().sum())
```

Построчный разбор

`df.duplicated()` — возвращает Series из булевых значений: `True` для каждой строки, которая полностью совпадает с какой-либо предыдущей строкой (по всем столбцам). Первое вхождение считается оригиналом (`False`), повторные — дубликатами (`True`).

`.sum()` — считает количество дубликатов.

Зачем удалять дубликаты:

- Дубликаты **смещают обучение**: если строка встречается 10 раз, модель будет считать её «важнее» остальных
- Увеличивают размер данных без увеличения информации
- Могут быть ошибкой при сборе данных (повторная запись)
- Удаление: `df.drop_duplicates(inplace=True)`

Результат: 0 дубликатов.

Ячейка 4: Распределение целевой переменной

```
plt.figure(figsize=(8, 4))
sns.histplot(df["Calories Burned"], bins=20)
plt.title("Распределение калорий")
plt.show()
```

Построчный разбор

```
plt.figure(figsize=(8, 4))
```

Создаёт новый объект Figure (холст для графика) с размерами 8 дюймов по ширине и 4 дюйма по высоте. Без этого вызова seaborn/matplotlib используют размер по умолчанию (6.4 × 4.8). Параметр `figsize` принимает кортеж (`width, height`).

Важно: именно `plt.figure()` с маленькой буквы `f`. `plt.Figure()` (с большой) создаёт объект Figure, но не регистрирует его в `pyplot` — размер не применится.

```
sns.histplot(df["Calories Burned"], bins=20)
```

Строит гистограмму распределения.

- **Первый аргумент** — `df["Calories Burned"]` — Series (столбец) с данными для визуализации. Квадратные скобки с именем столбца — стандартный способ доступа к столбцу DataFrame.
- **`bins=20`** — количество корзин (столбцов гистограммы). Каждая корзина охватывает интервал $(\max - \min) / \text{bins}$. 20 корзин — хороший баланс между детализацией и сглаживанием.

Зачем визуализировать целевую переменную:

- Понять диапазон значений (200–996 калорий)
- Оценить форму распределения: нормальное, скошенное, бимодальное?
- Линейная регрессия лучше работает при нормальном распределении у

- Обнаружить выбросы визуально

`plt.title("Распределение калорий")` — добавляет заголовок графика.

`plt.show()` — отображает график. В Jupyter Notebook без этой строки график обычно тоже отображается, но `plt.show()` гарантирует вывод и «закрывает» текущую фигуру, освобождая память.

Ячейка 5: Просмотр имён столбцов

```
print(df.columns)
```

`df.columns` — атрибут, возвращающий `Index` со списком имён всех столбцов. Полезно для:

- Точного написания имён (включая пробелы, скобки, регистр)
- Копирования имён для дальнейшего использования в коде
- Проверки, что все ожидаемые столбцы на месте

Ячейка 6: Кодирование пола и матрица корреляции

```
df["Gender"] = df["Gender"].map({"Male": 1, "Female": 0})
plt.figure(figsize=(10, 6))
sns.heatmap(df.corr(numeric_only=True), annot=True, fmt=".2f",
            cmap="coolwarm")
```

Построчный разбор

```
df["Gender"] = df["Gender"].map({"Male": 1, "Female": 0})
```

Кодируем категориальный признак `Gender` в числовой:

- `df["Gender"]` — доступ к столбцу `Gender`
- `.map(словарь)` — заменяет каждое значение в столбце согласно словарю. "Male" → 1, "Female" → 0. Значения, не найденные в словаре, станут NaN.
- `df["Gender"] = ...` — перезаписываем столбец преобразованным результатом

Почему `map`, а не `OneHotEncoder`?

Для бинарного признака (только 2 значения) достаточно одного числового столбца (0 и 1).

OneHotEncoder с `drop_first=True` даст тот же результат, но сложнее в коде. Для > 2 категорий — только ONE.

`plt.figure(figsize=(10, 6))` — холст 10×6 дюймов для более крупного графика.

`sns.heatmap(df.corr(numeric_only=True), annot=True, fmt=".2f", cmap="coolwarm")`

Строит тепловую карту корреляционной матрицы:

- `df.corr(numeric_only=True)` — вычисляет корреляцию Пирсона между всеми парами числовых столбцов. Возвращает квадратную матрицу (10×10) , где каждая ячейка — значение $r \in [-1, 1]$. Параметр `numeric_only=True` исключает нечисловые столбцы (в новых версиях pandas обязателен).
- `annot=True` — выводит числовые значения корреляции внутри каждой ячейки (аннотации)
- `fmt=".2f"` — формат аннотаций: два знака после запятой
- `cmap="coolwarm"` — цветовая палитра: синий для отрицательных корреляций, красный для положительных, белый для нулевых

Зачем корреляционная матрица:

- Найти признаки, сильно связанные с целевой переменной (потенциально полезные для модели)
- Обнаружить мультиколлинеарность (два признака с корреляцией > 0.9 — один можно убрать)
- Это один из ключевых графиков EDA

Ячейка 7: Генерация синтетических данных

```
np.random.seed(42)
n = 800

weight = np.random.randint(50, 100, n)
time = np.random.randint(30, 120, n)
speed = np.random.uniform(8, 15, n).round(1)
age = np.random.randint(18, 65, n)
heart_rate = np.random.randint(120, 175, n)
height = np.random.randint(155, 195, n)

calories = (
    0.6 * weight
    + 4.5 * time
```

```

+ 10 * speed
- 0.3 * age
+ 0.5 * heart_rate
+ np.random.normal(0, 30, n)
).astype(int)

synthetic = pd.DataFrame(
    {
        "Gender": np.random.randint(0, 2, n),
        "Age": age,
        "Height(cm)": height,
        "Weight(kg)": weight,
        "BMI": (weight / ((height / 100) ** 2)).round(1),
        "Running Time(min)": time,
        "Running Speed(km/h)": speed,
        "Distance(km)": (speed * time / 60).round(2),
        "Average Heart Rate": heart_rate,
        "Calories Burned": calories,
    }
)

df_new = pd.concat([df, synthetic], ignore_index=True)
print(df_new.shape)

```

Построчный разбор

`np.random.seed(42)`

Фиксирует «зерно» генератора случайных чисел. Все последующие вызовы `np.random.*` будут давать **одинаковые** результаты при каждом запуске кода. Число 42 — произвольное (часто используется в ML-сообществе как дань «Автостопом по Галактике»). **Зачем:** воспроизводимость — другой человек, запустив этот код, получит те же данные.

`n = 800`

Количество генерируемых синтетических строк. Выбрано так, чтобы синтетических данных было в 4 раза больше оригинальных (200), и они «задали тон» зависимостям.

`weight = np.random.randint(50, 100, n)`

Генерирует массив из `n` (800) случайных целых чисел от 50 (включительно) до 100 (не включительно). Это реалистичный диапазон веса человека в кг. Аналогично для `time`, `age`, `heart_rate`, `height`.

`speed = np.random.uniform(8, 15, n).round(1)`

`np.random.uniform(low, high, size)` — равномерное распределение вещественных

чисел от 8.0 до 15.0. `.round(1)` — округление до 1 знака после запятой (9.3, 12.7, ...). Скорость — непрерывная величина, поэтому `uniform` вместо `randint`.

Формула калорий:

```
calories = (  
    0.6 * weight      # тяжелее человек → больше калорий  
    + 4.5 * time      # дольше бег → больше калорий (главный фактор)  
    + 10 * speed      # быстрее бег → больше калорий  
    - 0.3 * age       # старше → чуть меньше калорий  
    + 0.5 * heart_rate # выше пульс → больше калорий  
    + np.random.normal(0, 30, n) # случайный шум  
) .astype(int)
```

- `np.random.normal(0, 30, n)` — нормальный шум со средним 0 и стандартным отклонением 30. Имитирует реальную случайность (факторы, не учтённые в модели). Без шума линейная модель дала бы $R^2 \approx 1.0$, что нереалистично.
- `.astype(int)` — приведение к целому числу (калории обычно считаются целыми)
- Коэффициенты (0.6, 4.5, 10, -0.3, 0.5) подобраны вручную, чтобы создать реалистичные зависимости

```
synthetic = pd.DataFrame({...})
```

Создаёт DataFrame из словаря, где ключи — имена столбцов, значения — массивы данных.

- `"BMI": (weight / ((height / 100) ** 2)).round(1) **` — BMI рассчитывается по формуле: $\text{вес(кг)} / \text{рост(м)}^2$. `height / 100` — перевод из сантиметров в метры.
- `"Distance(km)": (speed * time / 60).round(2)` — дистанция = скорость × время. Время в минутах → делим на 60 для получения часов → умножаем на скорость в км/ч → получаем км.
- `"Gender": np.random.randint(0, 2, n)` — случайный пол: 0 (Female) или 1 (Male)

```
df_new = pd.concat([df, synthetic], ignore_index=True)
```

Объединяет оригинальный и синтетический датасеты вертикально (строки друг под другом).

- `[df, synthetic]` — список DataFrames для объединения
- `ignore_index=True` — создаёт новый индекс 0, 1, 2, ... вместо сохранения старых (без этого было бы 0–199 и снова 0–799)

`print(df_new.shape)` → (1000, 10) — подтверждаем, что объединённый датасет содержит 200 + 800 = 1000 строк.

Ячейка 8: Корреляционная матрица объединённых данных

```
plt.figure(figsize=(10, 6))
sns.heatmap(df_new.corr(numeric_only=True), annot=True, fmt=".2f",
            cmap="coolwarm")
```

Код аналогичен ячейке 6, но строит матрицу для **объединённого** датасета `df_new`. Теперь корреляции значительно выше, потому что 800 синтетических строк имеют заложенную математическую зависимость. Это подтверждает, что данные готовы для обучения регрессионных моделей.

Ячейка 9: Разделение на обучающую и тестовую выборки

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score

X = df_new.drop(columns=["Calories Burned"])
y = df_new["Calories Burned"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(f"Train: {X_train.shape}")
print(f"Test: {X_test.shape}")
```

Построчный разбор

```
from sklearn.model_selection import train_test_split
```

Импортируем функцию разбиения данных из модуля `model_selection` библиотеки `scikit-learn`.

```
from sklearn.linear_model import LinearRegression
```

Импортируем класс линейной регрессии из модуля `linear_model`. Этот класс реализует OLS (метод наименьших квадратов).

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

Импортируем три функции метрик:

- `mean_absolute_error` — вычисляет MAE
- `mean_squared_error` — вычисляет MSE (RMSE = $\sqrt{\text{MSE}}$)
- `r2_score` — вычисляет коэффициент детерминации R^2

```
X = df_new.drop(columns=["Calories Burned"])
```

Создаём матрицу признаков X, удаляя целевую переменную из DataFrame.

- `drop(columns=[...])` — убирает указанные столбцы, возвращает новый DataFrame
- Результат: 9 столбцов-признаков (Gender, Age, Height, Weight, BMI, Running Time, Running Speed, Distance, Average Heart Rate)

```
y = df_new["Calories Burned"]
```

Извлекаем целевую переменную — Series с 1000 значениями калорий.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Разбиваем данные на обучающую и тестовую выборки:

- `X, y` — признаки и целевая переменная (передаются парой, чтобы соответствие строк сохранилось)
- `test_size=0.2` — 20% данных уходит в тест (200 строк), 80% в train (800 строк)
- `random_state=42` — фиксирует случайность разбиения. Каждый запуск даст одни и те же train/test наборы
- Возвращает 4 объекта: `X_train` (признаки для обучения), `X_test` (признаки для тестирования), `y_train` (ответы для обучения), `y_test` (ответы для проверки)

Почему 80/20? Стандартная пропорция для датасетов среднего размера. 80% (800 строк) — достаточно для обучения, 20% (200 строк) — для надёжной оценки метрик.

Ячейка 10: Масштабирование признаков

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
print("До Масштабирования")
print(X_train.describe().round(2))
print("После Масштабирования: ")
```

```
print(pd.DataFrame(X_train_scaled,  
columns=X_train.columns).describe().round(2))
```

Построчный разбор

```
from sklearn.preprocessing import StandardScaler
```

Импортируем StandardScaler — класс для Z-score стандартизации.

```
scaler = StandardScaler()
```

Создаём экземпляр StandardScaler. Пока он «пустой» — не знает ни μ , ни σ признаков.

```
X_train_scaled = scaler.fit_transform(X_train)
```

Два действия в одном вызове:

1. `fit(X_train)` — вычисляет μ (среднее) и σ (стандартное отклонение) для **каждого** из 9 столбцов по **обучающей** выборке. Эти значения сохраняются внутри объекта `scaler` (в атрибутах `scaler.mean_` и `scaler.scale_`).
2. `transform(X_train)` — применяет формулу $x_{std} = (x - \mu) / \sigma$ к каждому значению.

Результат: numpy-массив той же формы `(800, 9)`, но каждый столбец имеет среднее ≈ 0 и $std \approx 1$.

```
X_test_scaled = scaler.transform(X_test)
```

Только transform, НЕ fit_transform! Это критически важно:

- Используются μ и σ , вычисленные **по train** (в предыдущей строке)
- Тестовые данные масштабируются **теми же** параметрами
- Если бы мы сделали `fit_transform(X_test)` — это была бы **утечка данных (data leakage)**: μ и σ test set просочились бы в обработку

```
pd.DataFrame(X_train_scaled, columns=X_train.columns).describe().round(2)
```

StandardScaler возвращает numpy-массив (без имён столбцов). Оборачиваем его обратно в DataFrame с оригинальными именами для красивого вывода `.describe()`.

Что видим после масштабирования:

- **mean ≈ 0.00** для всех столбцов
 - **std ≈ 1.00** для всех столбцов
 - Значения теперь в диапазоне примерно от -2 до $+2$ (а не от 18 до 199 как было)
-

Ячейка 11: Обучение и оценка линейной регрессии

```
model = LinearRegression()
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)

mae = mean_absolute_error(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred) ** 0.5
r2 = r2_score(y_test, y_pred)

print(f"Mae: {mae:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"R2: {r2:.2f}")
```

Построчный разбор

`model = LinearRegression()`

Создаём объект линейной регрессии. Внутри реализован OLS (метод наименьших квадратов). Основные параметры (которые здесь используются по умолчанию):

- `fit_intercept=True` — автоматически вычисляет β_0 (свободный член)
- `copy_X=True` — не модифицирует входные данные
- `n_jobs=None` — один поток (для больших данных можно задать `n_jobs=-1`)

`model.fit(X_train_scaled, y_train)`

Обучение модели — подбор оптимальных коэффициентов β .

- **Что происходит внутри:** решается нормальное уравнение $\hat{\beta} = (X^T X)^{-1} X^T y$ (или его численно устойчивый эквивалент через SVD-разложение)
- **Вход:** `X_train_scaled` — масштабированные признаки (800, 9), `y_train` — целевая переменная (800,)
- **Результат:** внутри объекта `model` сохраняются:
 - `model.coef_` — массив из 9 коэффициентов $\beta_1, \dots, \beta_9$
 - `model.intercept_` — свободный член β_0

`y_pred = model.predict(X_test_scaled)`

Предсказание на тестовых данных.

- **Что происходит:** для каждой строки `X_test` вычисляется $\hat{y} = \beta_0 + \beta_1 \cdot x_1 + \dots + \beta_9 \cdot x_9$
- **Вход:** `X_test_scaled` — масштабированные тестовые признаки (200, 9)
- **Выход:** numpy-массив из 200 предсказанных значений калорий

```
mae = mean_absolute_error(y_test, y_pred)
```

Вычисляет $MAE = (1/n) \cdot \sum |y_i - \hat{y}_i|$. Принимает два массива одинаковой длины: реальные значения и предсказания.

```
`rmse = mean_squared_error(y_test, y_pred) 0.5 ** mean_squared_error
```

возвращает MSE. Возводим в степень 0.5 (= берём корень) чтобы получить RMSE.

Альтернатива: `mean_squared_error(y_test, y_pred, squared=False)` в новых версиях `sklearn` (но в нашем коде используется ручное извлечение корня — `** 0.5` — это возведение в степень 1/2).

```
r2 = r2_score(y_test, y_pred)
```

Вычисляет $R^2 = 1 - SS_{res}/SS_{tot}$. Принимает те же два массива.

```
print(f"Mae: {mae:.2f}")
```

f-строка (format string) Python 3.6+: `{mae:.2f}` подставляет значение `mae` с 2 знаками после запятой.

Ячейка 12: График «Реальные vs Предсказанные»

```
plt.figure(figsize=(10, 5))
plt.scatter(y_test, y_pred, alpha=0.5, color="green")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], "r--",
         linewidth=2)
plt.xlabel("Реальные калории")
plt.ylabel("Предсказанные калории")
plt.title("Реальные vs Предсказанные")
plt.show()
```

Построчный разбор

```
plt.scatter(y_test, y_pred, alpha=0.5, color="green")
```

Рисует точечный график (scatter plot):

- `y_test` — значения по оси X (реальные калории)
- `y_pred` — значения по оси Y (предсказанные калории)
- `alpha=0.5` — прозрачность точек (0 = невидимые, 1 = непрозрачные). 0.5 позволяет видеть области, где точки накладываются друг на друга
- `color="green"` — зелёный цвет точек

```
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], "r--",
         linewidth=2)
```

Рисует пунктирную красную линию — **идеальное предсказание** ($y_{pred} = y_{test}$):

- Первый аргумент — координаты X: [минимум, максимум]
- Второй аргумент — координаты Y: [минимум, максимум] (те же)
- "r--" — красная (r) пунктирная (--) линия
- linewidth=2 — толщина линии

Интерпретация графика: если бы модель предсказывала идеально, все точки лежали бы на красной линии. Чем дальше точки от линии — тем хуже предсказание. Можно визуально оценить:

- Есть ли систематическое смещение (точки выше или ниже линии)
- В каком диапазоне модель ошибается больше всего
- Есть ли группы точек с аномально большими ошибками

Ячейка 13: График остатков (Residuals)

```
residuals = y_test - y_pred
plt.figure(figsize=(8, 6))
plt.scatter(y_pred, residuals, alpha=0.5, color="blue")
plt.axhline(y=0, color="r", linewidth=2, linestyle="--")
plt.xlabel("Предсказанные калории")
plt.ylabel("Остатки (ошибка)")
plt.title("График остатков (Residuals)")
plt.show()
```

Построчный разбор

```
residuals = y_test - y_pred
```

Вычисляет остатки: $e_i = y_i - \hat{y}_i$ для каждого из 200 тестовых наблюдений.

Положительный остаток — модель занизила, отрицательный — завысила.

```
plt.scatter(y_pred, residuals, ...)
```

Scatter plot: по оси X — предсказанные значения, по оси Y — остатки.

```
plt.axhline(y=0, color="r", linewidth=2, linestyle="--")
```

Рисует горизонтальную линию на уровне $y=0$:

- **axhline** — «axis horizontal line», горизонтальная линия через всю ширину графика
- Эта линия — «нулевая ошибка», идеальное предсказание

Что искать на графике остатков:

- **Случайное облако вокруг нуля** — хорошо, модель работает адекватно

- **Воронка** (расширение при росте $\hat{y}$) — гетероскедастичность, нарушение допущения OLS
 - **Дуга** — нелинейная зависимость, линейная модель не справляется
 - **Кластеры** — разные группы данных, которые модель обрабатывает по-разному
-

Ячейка 14: Диаграмма значимости признаков

```
coefficient = pd.DataFrame(  
    {"Признак": X_train.columns, "Коэффициент": model.coef_  
}).sort_values("Коэффициент", ascending=False)  
plt.figure(figsize=(8, 6))  
sns.barplot(x="Коэффициент", y="Признак", data=coefficient,  
palette="coolwarm")  
plt.title("Значимость признаков — Линейная регрессия")  
plt.axvline(x=0, color="black", linestyle="--")  
plt.show()
```

Построчный разбор

```
pd.DataFrame({"Признак": X_train.columns, "Коэффициент": model.coef_})
```

Создаём DataFrame из двух столбцов:

- `X_train.columns` — имена 9 признаков
- `model.coef_` — массив из 9 коэффициентов $\beta_1, \dots, \beta_9$, подобранных моделью

```
.sort_values("Коэффициент", ascending=False)
```

Сортировка по убыванию коэффициента. Самые «положительные» признаки сверху, самые «отрицательные» — снизу.

```
sns.barplot(x="Коэффициент", y="Признак", data=coefficient, palette="coolwarm")
```

Горизонтальная столбчатая диаграмма:

- `x="Коэффициент"` — длина столбца = значение коэффициента
- `y="Признак"` — подпись каждого столбца
- `palette="coolwarm"` — цветовая палитра: тёплые цвета для положительных, холодные для отрицательных

```
plt.axvline(x=0, color="black", linestyle="--")
```

Вертикальная линия на $x=0$. Разделяет положительные и отрицательные коэффициенты.

Важно о интерпретации: поскольку признаки масштабированы через `StandardScaler`, коэффициенты показывают **относительную значимость** — изменение предсказания при

изменении признака на одно стандартное отклонение. Нельзя говорить «каждый лишний км добавляет 4 калории» — это было бы верно только для нестандартизированных признаков.

Ячейка 15: Проверка переобучения линейной модели

```
y_pred_train = model.predict(X_train_scaled)
r2_train = r2_score(y_train, y_pred_train)
r2_test = r2_score(y_test, y_pred)
print(f"R2 Train: {r2_train:.2f}")
print(f"R2 Test: {r2_test:.2f}")
```

Построчный разбор

```
y_pred_train = model.predict(X_train_scaled)
```

Предсказываем калории на **обучающей** выборке (тех же данных, на которых модель училась).

```
r2_train = r2_score(y_train, y_pred_train)
```

R^2 на train. Если модель переобучена, этот показатель будет значительно выше, чем R^2 на test.

```
r2_test = r2_score(y_test, y_pred)
```

R^2 на test (уже вычислен ранее).

Интерпретация:

- R^2 Train = 0.44, R^2 Test = 0.39 → разница 0.05
- Разница маленькая → **переобучения нет**
- Модель одинаково (посредственно) работает и на train, и на test
- Если бы R^2 Train = 0.95, R^2 Test = 0.40 → серьёзное переобучение

Ячейка 16: Проверка на одном примере

```
sample = X_test_scaled[0]
real = y_test.iloc[0]
predicted = model.predict([sample])[0]

print(f"Реальные калории: {real}")
```



```
print(f"Предсказанные калории: {predicted:.2f}")
print(f"Ошибка: {abs(real-predicted):.2f}")
```

Построчный разбор

```
sample = X_test_scaled[0]
```

Берём первую строку тестовой выборки (масштабированную) — это вектор из 9 признаков одного человека.

```
real = y_test.iloc[0]
```

Реальное количество калорий для этого человека. Используем `.iloc[0]` потому что `y_test` — это Series с оригинальными индексами из DataFrame (не обязательно 0, 1, 2...). `.iloc` — позиционный доступ (по номеру позиции, а не по метке индекса).

```
predicted = model.predict([sample])[0]
```

- `[sample]` — оборачиваем одномерный массив в двумерный (модель ожидает 2D-массив: строки × столбцы)
- `model.predict(...)` — возвращает массив предсказаний (в данном случае из 1 элемента)
- `[0]` — извлекаем единственное предсказание из массива

`abs(real-predicted)` — абсолютная ошибка для этого конкретного наблюдения.

Ячейка 17: Итоговая таблица линейной регрессии

```
results = pd.DataFrame(
    {
        "Метрики": ["MAE", "RMSE", "R2"],
        "Значение": [
            f"{mean_absolute_error(y_test, y_pred):.2f}",
            f"{mean_squared_error(y_test, y_pred)**0.5:.2f}",
            f"{r2_score(y_test, y_pred):.2f}",
        ],
    },
)
print(results)
```

Создаёт компактную таблицу с тремя метриками для удобного представления результатов линейной модели. Функции метрик вызываются повторно (можно было бы использовать ранее вычисленные переменные `mae`, `rmse`, `r2`).

Ячейка 18: Полиномиальная регрессия

```
from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly = poly.fit_transform(X_train_scaled)
X_test_poly = poly.transform(X_test_scaled)
print(f"Признаков до: {X_test_scaled.shape[1]}")
print(f"Признаков после: {X_test_poly.shape[1]}")

model_poly = LinearRegression()
model_poly.fit(X_train_poly, y_train)
y_pred_poly = model_poly.predict(X_test_poly)
print("Полиномиальная регрессия")
print(f"MAE: {mean_absolute_error(y_test, y_pred_poly):.2f}")
print(f"RMSE: {mean_squared_error(y_test, y_pred_poly)**0.5:.2f}")
print(f"R2: {r2_score(y_test, y_pred_poly):.2f}")
```

Построчный разбор

```
from sklearn.preprocessing import PolynomialFeatures
```

Импортируем класс для создания полиномиальных признаков.

```
poly = PolynomialFeatures(degree=2, include_bias=False)
```

Создаём трансформер:

- **degree=2** — максимальная степень полинома. Создаёт: оригинальные признаки, их квадраты (x_1^2 , x_2^2 , ...) и все попарные произведения ($x_1 \cdot x_2$, $x_1 \cdot x_3$, ...).
- **include_bias=False** — не добавлять столбец из единиц (bias/intercept).
LinearRegression сам добавляет intercept через параметр `fit_intercept=True`.

```
X_train_poly = poly.fit_transform(X_train_scaled)
```

- **fit** — запоминает количество и имена входных признаков
- **transform** — создаёт полиномиальные комбинации

Из 9 признаков создаётся 54:

- 9 оригинальных: x_1 , x_2 , ..., x_9
- 9 квадратов: x_1^2 , x_2^2 , ..., x_9^2
- $C(9,2) = 36$ произведений: x_1x_2 , x_1x_3 , ..., x_8x_9
- Итого: $9 + 9 + 36 = 54$

```
X_test_poly = poly.transform(X_test_scaled)
```

Только `transform` — те же полиномиальные комбинации для тестовых данных.

```
model_poly = LinearRegression()
```

Создаём **новый** объект `LinearRegression`. Это важно — не переиспользуем старый `model`, чтобы не путать результаты.

```
model_poly.fit(X_train_poly, y_train)
```

Обучаем на 54 полиномиальных признаках. Модель подбирает 54 коэффициента + `intercept`.

Результат: $R^2 = 0.37$ — **хуже** линейной модели (0.39). Больше признаков не значит лучше!

Ячейка 19: Проверка переобучения полиномиальной модели

```
y_pred_poly_train = model_poly.predict(X_train_poly)
r2_poly_train = r2_score(y_train, y_pred_poly_train)
r2_poly_test = r2_score(y_test, y_pred_poly)
print(f"R2 Train: {r2_poly_train:.2f}")
print(f"R2 Test: {r2_poly_test:.2f}")
```

Результат: R^2 Train = 0.55, R^2 Test = 0.37. Разница = 0.18 — значительное переобучение.

Модель «запомнила» шум в обучающих данных. 54 признака дают ей слишком много «степеней свободы» — она подстроилась под тренировочные данные, но не обобщает на новых.

Ячейка 20: Ridge-регуляризация

```
from sklearn.linear_model import Ridge, Lasso

ridge = Ridge(alpha=1)
ridge.fit(X_train_poly, y_train)
y_pred_ridge = ridge.predict(X_test_poly)
print(f"Ridge R² Train: {r2_score(y_train,
    ridge.predict(X_train_poly)):.2f}")
print(f"Ridge R² Test: {r2_score(y_test, y_pred_ridge):.2f}")
```

Построчный разбор

```
from sklearn.linear_model import Ridge, Lasso
```

Импортируем классы Ridge (L2) и Lasso (L1) регуляризации.

```
ridge = Ridge(alpha=1)
```

Создаём Ridge-модель:

- **alpha=1** — сила регуляризации (λ в формуле). $\alpha=0$ — обычная линейная регрессия, $\alpha \rightarrow \infty$ — все коэффициенты $\rightarrow 0$.
- Значение $\alpha=1$ — default, часто слишком слабое для сильного переобучения.

```
ridge.fit(X_train_poly, y_train)
```

Обучение Ridge на 54 полиномиальных признаках. Внутри решается:

$$\hat{\beta} = (X^T X + \alpha I)^{-1} X^T y$$

Матрица αI стабилизирует $X^T X$ и уменьшает коэффициенты.

Результат: R^2 Train = 0.55, R^2 Test = 0.37 — **не помог**. При $\alpha=1$ штраф недостаточно сильный. Для улучшения нужно подобрать оптимальный α через кросс-валидацию (например, `RidgeCV`).

Ячейка 21: Lasso-регуляризация

```
lasso = Lasso(alpha=1)
lasso.fit(X_train_poly, y_train)
y_pred_lasso = lasso.predict(X_test_poly)
print(f"Lasso R² Train: {r2_score(y_train,
lasso.predict(X_train_poly)):.2f}")
print(f"Lasso R² Test: {r2_score(y_test, y_pred_lasso):.2f}")
```

Построчный разбор

```
lasso = Lasso(alpha=1)
```

Создаём Lasso-модель (L1-регуляризация) с $\alpha=1$.

Что делает Lasso внутри: решает задачу оптимизации $\sum (y_i - \hat{y}_i)^2 + \alpha \cdot \sum |\beta_j|$ методом coordinate descent (координатный спуск, итеративный алгоритм, потому что L1-норма недифференцируема в нуле).

Результат: R^2 Train = 0.55, R^2 Test = 0.39 — **чуть лучше** Ridge и полиномиальной. Lasso обнулil часть из 54 полиномиальных коэффициентов, оставив только значимые. Это снизило переобучение и вернуло R^2 Test к уровню линейной модели.

Ячейка 22: Сравнительная таблица всех моделей

```
comparison = pd.DataFrame({
    "Модель": ["Линейная регрессия", "Полиномиальная (degree=2)", "Ridge (alpha=1)", "Lasso (alpha=1)"],
    "R2 Train": [
        f"{r2_score(y_train, model.predict(X_train_scaled)):.2f}",
        f"{r2_score(y_train, model_poly.predict(X_train_poly)):.2f}",
        f"{r2_score(y_train, ridge.predict(X_train_poly)):.2f}",
        f"{r2_score(y_train, lasso.predict(X_train_poly)):.2f}",
    ],
    "R2 Test": [
        f"{r2_score(y_test, y_pred):.2f}",
        f"{r2_score(y_test, y_pred_poly):.2f}",
        f"{r2_score(y_test, y_pred_ridge):.2f}",
        f"{r2_score(y_test, y_pred_lasso):.2f}",
    ],
    "MAE": [...],
    "RMSE": [...],
})
print(comparison.to_string(index=False))
```

Финальная таблица, объединяющая метрики **всех четырёх моделей** для сравнения. Каждая строка — одна модель, столбцы — R^2 Train, R^2 Test, MAE, RMSE. Метод `.to_string(index=False)` выводит таблицу без индексов строк (0, 1, 2, 3) для чистого вида.

ЧАСТЬ IV: КЛЮЧЕВЫЕ ВЫВОДЫ

Что мы сделали

1. **EDA:** загрузили данные (200 строк, 10 столбцов), проверили пропуски и дубликаты — данные чистые
2. **Обнаружили проблему:** исходные данные не имеют реальных зависимостей (корреляции ≈ 0.10)

3. **Решили проблему:** сгенерировали 800 синтетических строк с математической формулой зависимости калорий от веса, времени, скорости и пульса
4. **Построили 4 модели:** линейная, полиномиальная, Ridge, Lasso
5. **Обнаружили переобучение** у полиномиальной модели (R^2 Train = 0.55, R^2 Test = 0.37)
6. **Применили регуляризацию:** Lasso справился лучше Ridge, вернув R^2 Test до 0.39

Итоговая таблица

Модель	R^2 Train	R^2 Test	Разница	Переобучение?
Линейная	0.44	0.39	0.05	Нет
Полиномиальная	0.55	0.37	0.18	Да
Ridge (alpha=1)	0.55	0.37	0.18	Да (не помог)
Lasso (alpha=1)	0.55	0.39	0.16	Частично исправлено

Главные уроки

1. **Больше признаков $\neq$ лучше.** Полиномиальная модель (54 признака) работает хуже линейной (9 признаков) из-за переобучения.
2. **Lasso эффективнее Ridge** когда среди многих признаков только часть реально важна — он «выключает» лишние.
3. **Масштабирование обязательно** до регуляризации и при сравнении значимости признаков.
4. **Data leakage предотвращается** правильным порядком: сначала split, потом fit scaler на train.
5. **Всегда проверяй переобучение** сравнением R^2 на train и test.